

Simulating LRU in Software

1 Core Idea

Problem

Exact LRU needs expensive hardware or per-reference list updates. Software algorithms approximate LRU using periodic page-usage information.

Two software approaches are important:

- **NFU (Not Frequently Used)**: counts how often each page was referenced.
- **Aging**: improves NFU by making old references gradually lose importance.

2 NFU Algorithm

Definition

The **Not Frequently Used (NFU)** algorithm gives each page a software counter. At every clock interrupt, the page's R bit is added to its counter.

Each page counter starts at 0. On every clock tick:

$$\text{counter}[p] \leftarrow \text{counter}[p] + R[p]$$

When a page fault occurs, NFU evicts the page with the lowest counter.

Counter value	Meaning
High	Page has been referenced many times in the past.
Low	Page has been referenced fewer times.

3 Main Problem with NFU

NFU Does Not Forget

NFU keeps adding to counters but does not reduce old counts. A page heavily used long ago can look important forever.

Example: in a multipass compiler, pages used heavily in pass 1 may keep high counters even after pass 1 is finished. Later passes may need different pages, but NFU may still protect the old pass-1 pages.

Problem

NFU measures frequency, but it does not measure recency well.

4 Aging Algorithm

Definition

Aging is a software approximation of LRU that shifts each page's counter right at every clock tick and inserts the current R bit into the leftmost bit.

Aging modifies NFU in two ways:

1. Shift each counter right by 1 bit.
2. Insert the page's current R bit into the leftmost bit.

Aging Update

For an n -bit counter:

$$\text{counter}[p] \leftarrow (R[p] \ll (n - 1)) + (\text{counter}[p] \gg 1)$$

For an 8-bit counter, a referenced page gets a leading 1. A page not referenced gets a leading 0.

5 How the R Bit Changes

Important

The R bit changes because of page references during a clock interval. It is not produced by the aging counter.

For each page:

1. At the start of a clock interval, the OS has cleared R to 0.
2. If the CPU reads or writes that page during the interval, hardware sets $R = 1$.
3. If the page is not touched during the interval, R stays 0.
4. At the next clock tick, the OS reads R , updates the aging counter, then clears R back to 0 for the next interval.

R Bit Life Cycle

OS clears $R \rightarrow$ CPU may set $R \rightarrow$ clock tick samples $R \rightarrow$ OS clears R again

Meaning of the Top Row in the PNG

Each top row, such as 1,0,1,0,1,1, is a record of which pages were touched during that clock interval. It is not calculated from the previous top row.

Six R Values for Six Pages

The figure has six pages, numbered 0 through 5. Therefore each clock tick shows six R values, one for each page.

Page	0	1	2	3	4	5
R bits at one tick	1	0	1	0	1	1

This means:

Page	Meaning
Page 0	$R = 1$, so page 0 was referenced in this interval.
Page 1	$R = 0$, so page 1 was not referenced in this interval.
Page 2	$R = 1$, so page 2 was referenced in this interval.
Page 3	$R = 0$, so page 3 was not referenced in this interval.
Page 4	$R = 1$, so page 4 was referenced in this interval.
Page 5	$R = 1$, so page 5 was referenced in this interval.

No Direct Link Between Consecutive R Rows

The next set of R values is a fresh measurement for the next clock interval. It is not computed from the previous R values. It depends only on which pages the CPU touched during that next interval.

For example:

Page	0	1	2	3	4	5
Tick 0 R	1	0	1	0	1	1
Tick 1 R	1	1	0	0	1	0

Tick 1's row means pages 0, 1, 4 were referenced during the next interval. It does not mean the OS transformed tick 0's row into tick 1's row.

6 Aging Figure

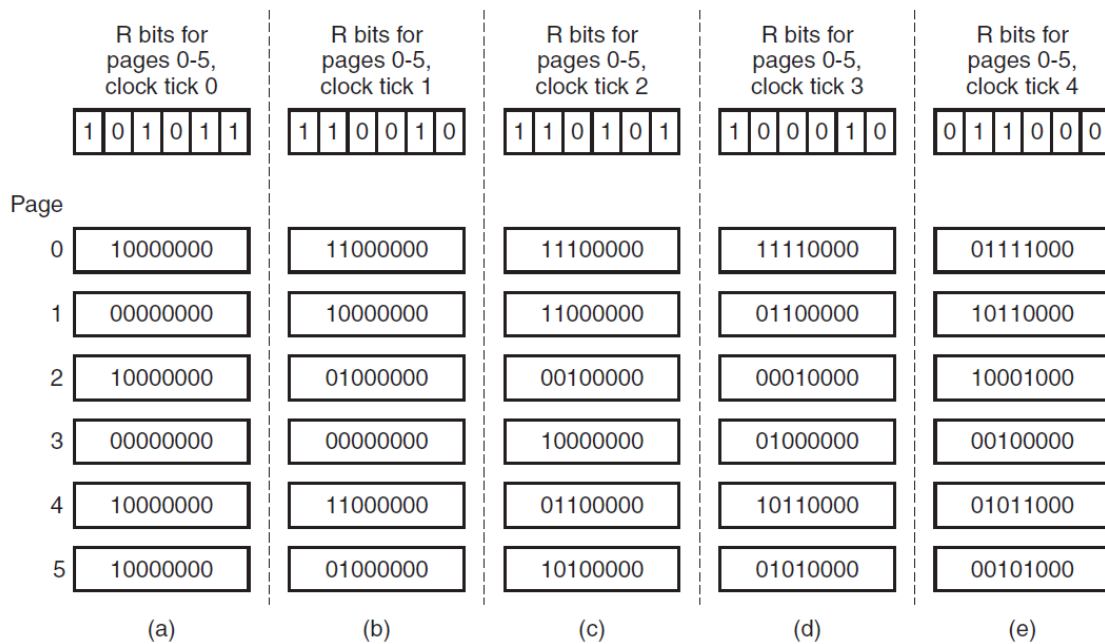


Figure 3-17. The aging algorithm simulates LRU in software. Shown are six pages for five clock ticks. The five clock ticks are represented by (a) to (e).

How to Read the Figure

Each column is one clock tick. The row for each page shows its aging counter after shifting right and inserting that tick's *R* bit on the left.

Detailed Reading

The figure tracks six pages: 0, 1, 2, 3, 4, 5. Each page has an 8-bit aging counter.

Part of figure	Meaning
Top row of each column	The <i>R</i> bits for pages 0 through 5 during that clock interval.
Rows labeled 0 to 5	The aging counters for pages 0 through 5.
Leftmost bit of a counter	Whether the page was referenced in the most recent interval.
Bits farther right	Older reference history. Each tick shifts old history one position to the right.

Column (a): After Clock Tick 0

The *R* bits are:

1, 0, 1, 0, 1, 1

So pages 0, 2, 4, 5 were referenced. Their counters get a leftmost 1:

Page	Counter after tick 0
0	10000000
1	00000000
2	10000000
3	00000000
4	10000000
5	10000000

Column (b): After Clock Tick 1

Now the R bits are:

1, 1, 0, 0, 1, 0

Before inserting these bits, every old counter shifts right. For example:

page 0 : $10000000 \gg 1 = 01000000$

Since page 0 has $R = 1$, insert 1 at the left:

$01000000 \rightarrow 11000000$

Page 2 has $R = 0$, so its old counter shifts right and gets a leftmost 0:

$10000000 \gg 1 = 01000000$

That is why page 2's counter becomes 01000000.

What the Final Column Means

In column (e), the counters are:

Page	Counter
0	01111000
1	10110000
2	10001000
3	00100000
4	01011000
5	00101000

The smallest counter is the best victim because it shows the weakest recent-use pattern.

Page	Interpretation from column (e)
1 and 2	Large leftmost bits, so they were referenced recently. Poor victims.
0 and 4	Some recent history, but not the smallest counters.
3 and 5	Both have small counters, so they are older candidates.

Victim from the Figure

In column (e), page 3 has counter 00100000, while page 5 has 00101000. Since $00100000 < 00101000$, aging chooses page 3.

Why This Is Not Exact LRU

Pages 3 and 5 were both not referenced for the last two ticks. Aging cannot know the exact order of references inside an earlier clock interval, because it records only one R bit per interval.

7 One Counter Update Example

Problem

An 8-bit aging counter is:

01010000

At the next clock tick, the page has $R = 1$. What is the new counter?

Solution

First shift right:

$$01010000 \gg 1 = 00101000$$

Then insert $R = 1$ into the leftmost bit:

$$00101000 \rightarrow 10101000$$

Answer

The new aging counter is:

10101000

8 Choosing a Victim

When a page fault occurs, aging chooses the page with the lowest counter.

Victim Rule

victim = page with the smallest aging counter

A lower counter means the page has not been referenced recently, or has been referenced less often in recent clock intervals.

9 Why Aging Approximates LRU

Recent references are stored in high-order bits, so they matter more than old references.

Bit position	Meaning
Leftmost bits	Recent clock ticks; most important.
Rightmost bits	Older clock ticks; less important.
All zeros	Page has not been referenced during the tracked history window.

Aging Effect

Old references fade because the counter shifts right on every clock tick.

10 How Aging Differs from True LRU

Aging is close to LRU, but it is not exact.

Difference	Explanation
Only one bit per interval	Aging knows whether a page was referenced during a clock tick, but not exactly when inside that interval.
Finite counter size	With 8 bits, only the last 8 clock intervals are represented. Older history is lost.

Limited Horizon

If two pages both have counter 00000000, aging cannot tell whether one was last used 9 ticks ago and the other 1000 ticks ago.

11 Why 8 Bits Can Be Enough

If the clock tick is about 20 msec and the counter has 8 bits, the tracked window is:

$$8 \times 20 \text{ msec} = 160 \text{ msec}$$

Practical Interpretation

If a page has not been referenced for about 160 msec, it is often a reasonable replacement candidate.

12 NFU vs Aging

Algorithm	Main idea	Main weakness
NFU	Add R to a counter at each clock tick.	Old references remain forever.
Aging	Shift right, then insert R at the leftmost bit.	Approximate only; loses exact order inside clock intervals.

13 Final Summary

One-Box Summary

NFU approximates use frequency with counters.
NFU fails because it does not forget old references.
Aging shifts counters right and inserts R at the left.
Aging approximates LRU by making recent references count more.